Real-Time Signal Processing Using FPGAs

Funded and Sponsored By Idaho National Laboratory

Developed at Loyola Marymount University

Presented by

Stephen Ude

Matthew Shao

Evan Mow

Meet the team

Matthew Shao

Electrical Engineer
Minor in Computer Science

Stephen Ude

Electrical / Computer Engineer

Evan Mow

Electrical / Computer Engineer

Idaho National Laboratory

Content

1 Project Background

2 Industry Relevance

3 System Design & Implementation

4 Project Milestones & Scope

5 Future Prospects

6 Sources

1 Project Background (1)

DAQ collects sensor data from 145 kV power line at 14 Ms/s

Sensor head transmits data by laser to metal pad, then relays to computer

Data extracted to hard drives in 2.5 GB batches, accumulating hundreds of TBs

Process mass sensor data in real-time with 1 Hz spectral and 1 μs temporal resolutions

Primary Capstone Focus

Apply or develop feature extraction techniques

Capstone Adjacent/Extension

Happy Customer!!!

(and we graduate!!!)

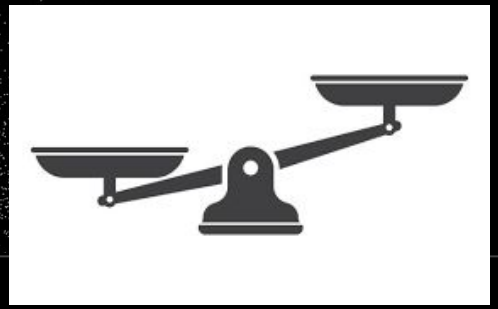
Project Background (2)

Limitations Overview:

- Traditional PCs are too slow to process large sensor datasets at this scale
- Supercomputers are unaffordable for your average customer

Capstone Approach:

- Implement signal processing on Field-Programmable Gate Array (FPGA) boards
- Achieve 1 Hz spectral resolution with adjustable temporal resolution (starting at 1 μs) at 14 Ms/s



Project Proposals (3)

Proposed Signal Processing Algorithms:

- Sliding Window Infinite Fourier Transform (SWIFT) algorithm
- Fast Fourier Transform (FFT) algorithm

Proposed FPGA Boards:

- ZCU102 FPGA Board for SWIFT algorithm
- VPK180 FPGA Board for both SWIFT and FFT algorithms

Evaluation of SWIFT vs. FFT

Why SWIFT [1]?

Pros:

- Recurrent structure reduces consumption of hardware resources
- Simple architecture allows for straightforward FPGA implementation
- FFT batch processing adds latency equal to the window length (the number of samples seen on the spectrogram)

Cons:

- Entirely dependent on prior bin computations, leading to rounding errors in later steps
- Highly susceptible to noise and precision loss requiring additional pre or post-processing to clean outputs

Evaluation of SWIFT vs. FFT

Why FFT [2]?

Pros:

- Well-established algorithm with abundant online resources and reference implementations
- Supports high levels of parallelism, enabling faster computation time
- Mathematically stable, even when processing large datasets

Cons:

- Consumes significantly more hardware on FPGA than SWIFT
- Real-time throughput is difficult for continuous multi-MHz streams, especially with datasets as large as 7 million samples
- Requires power-of-two input sizes
  - Padding the data length, which decreases accuracy
  - Splitting into smaller blocks, which significantly increases code complexity

1

Figure 1: ZCU102 FPGA Board

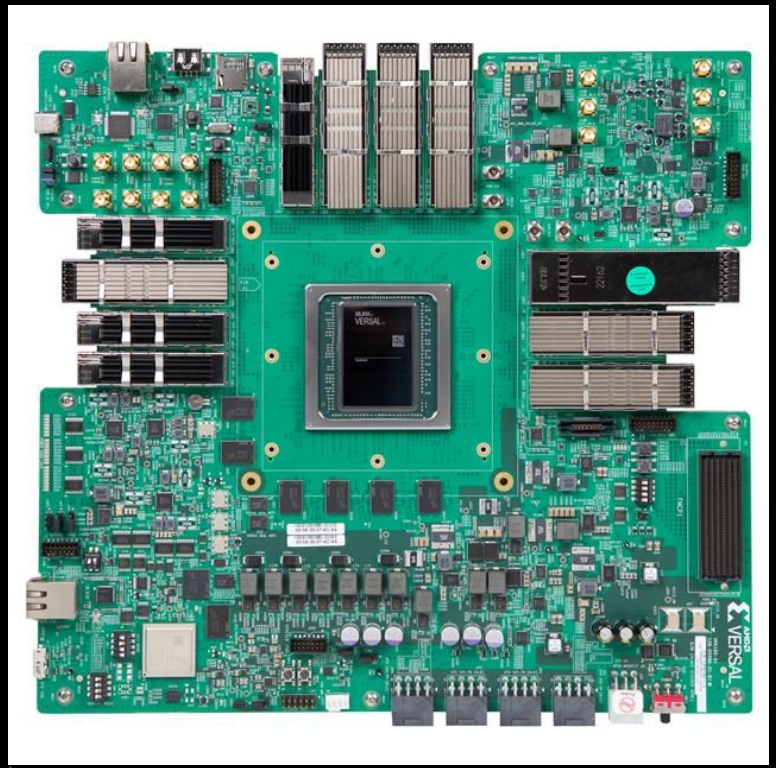


Figure 2: VPK180 FPGA Board

2

Industry Relevance

1

Traditional CPU and GPU methods cannot efficiently handle continuous multi-MHz data streams.

2

Compared to CPU and GPU implementations, FPGA-based signal processing achieves significantly lower latency and higher throughput.

3

FPGAs offer a cost-effective and powerful alternative to supercomputers for high-throughput signal processing.

3

System Design & Implementation

The implementation of SWIFT and FFT algorithms on the FPGA boards is structured into a three-phase development process:

1

IP Block Creation (SWIFT & FFT)

- Use Vitis HLS for algorithm design
- Written in synthesizable C/C++
- Auto compiles C/C++ code to RTL languages

2

Hardware Platform Integration

- Insert IP blocks into Vivado block design
- Combine with existing IPs for system build
- Used as platform for software applications

3

Data Transfer & Storage

- Implement DMA logic in Vitis Unified
- Programmed in Baremetal C
- Enables PS to PL communication including storage elements

4

Project Milestones & Scope

Milestone #1:
Implement SWIFT algorithm on ZCU102 FPGA board

Milestone #2:
Implement SWIFT algorithm on VPK180 FPGA board

Milestone #3:
Implement FFT algorithm on VPK180 FPGA board

Milestone #4:
Create Feature extraction using machine learning

5

Future Prospects

Next Steps:

1. Optimize FPGA architectures to balance throughput, accuracy, and resource usage for each algorithm
2. Investigate effects of FFT and SWIFT on sensor data output to see differences
3. Integrate machine learning algorithms for adaptive pre or post-processing to reduce noise and enhance spectral quality
4. Scale system design for deployment in real-world high-voltage-high-current monitoring environments

Sources

Key References:

[1] Grado, Logan L, et al. "The Sliding Windowed Infinite Fourier Transform [Tips & Tricks]." *IEEE Signal Processing Magazine*, vol. 34, no. 5, 1 Sept. 2017, pp. 183–188, par.nsf.gov/biblio/10078055-sliding-windowed-infinite-fourier-transform-tips-tricks, https://doi.org/10.1109/msp.2017.2718039. Accessed 21 July 2025.

[2] Oberst, Ulrich. "The Fast Fourier Transform." *SIAM Journal on Control and Optimization*, vol. 46, no. 2, Jan. 2007, pp. 496–540, https://doi.org/10.1137/060658242.

Acknowledgments

We would like to thank **Bjorn Vaageensmith, Ph.D.** and **Jesse Reeves, Ph.D.** at Idaho National Laboratory for sponsoring our capstone project and mentorship

We would like to thank **Lei Huang, Ph.D.** at Loyola Marymount University for her continuous support through our capstone

THANK YOU!

QUESTIONS?

Presented by

Stephen Ude

Matthew Shao